Pattern Ricorsione: Backtracking

Quando usarlo: Devi esplorare sistematicamente un albero di decisioni per trovare una sequenza ottimale, tutte le combinazioni valide o il percorso d'uscita da un labirinto (es. N-Regine, Anagrammi).

Lezioni di riferimento: Giovedì (N-Regine, Anagrammi).

Caratteristica chiave: Si porta avanti uno "stato" (es. una lista parziale). Si sporca lo stato provando una mossa, si esplora in profondità e, in caso di fallimento o fine del ramo, si "pulisce" lo stato (backtrack).

Template Python (Esplorazione Sistematica)

```
def exploration(self, partial_sol, parametri_extra):

    # 🚨 1. CASO BASE (Condizione di Terminazione)
    # L'obiettivo è raggiunto? (es. ho piazzato N regine)
    if condizione_di_fine_raggiunta(partial_sol):

        # Verifica finale ed eventuale salvataggio in memoria globale
        if self.solution_is_good(partial_sol):
            self.lista_soluzioni.append(partial_sol.copy()) # 🚨 IL .copy() È
VITALE!

        return # Torna indietro senza esplorare oltre

    else:
        # 2. ESPLORAZIONE DELLE SCELTE (Ciclo For)
        # Provo tutte le mosse possibili a questo livello del problema
        for mossa in mosse_disponibili():

            # 🚨 3. SPORCO LO STATO (Aggiungo la mossa)
            partial_sol.append(mossa)

            # 🚨 4. PRUNING (Filtro anti-spreco)
            # Se la mossa genera un percorso invalido, evito di scendere!
            if self.mossa_is_acceptable(partial_sol):

                # 5. RICORSIONE IN PROFONDITÀ (Scendo di un livello)
                self.exploration(partial_sol, parametri_extra)


            partial_sol.pop()
```

```
# 🚨 6. BACKTRACKING (Smonto la soluzione / Pulisco lo stato)
            # Tolgo SEMPRE l'ultima mossa, sia per fallimento che per ripartire!
```

Note d'Esame

- **Il `.pop()` è obbligatorio:** Senza questa operazione di "pulizia", lo stato accumulerà in memoria un'infinita spazzatura di mosse scartate, bloccando l'algoritmo.
- **Attenzione al `.copy()`:** Quando salvi una soluzione (fase 1), devi usare `.copy()`. Se aggiungi direttamente la lista `partial_sol`, a causa dei `.pop()` successivi, a fine esecuzione troverai solo liste vuote.
- **Il potere del Pruning (Fase 4):** Verificare una mossa *prima* di scendere in profondità taglia i rami morti alla radice, riducendo un problema da milioni di calcoli a poche centinaia. È la differenza tra superare l'esame e mandare il PC in crash.